

Week-3 Module

Q1 A university's Training and Placement cell have successfully completed its annual placement drive. The placement department has collected the salary package (in LPA) offered to students by different companies. However, the records were entered in the order they received, resulting in an unsorted list of package values.

Before publishing the placement statistics report, the department wants all package values arranged in ascending order. Since the university plans to handle placement data for thousands of students in the coming years, they want a sorting technique that is efficient for large datasets.

Your task is to implement the Merge sort Algo to sort the package values in increasing order. Merge sort follows a divide-and-conquer approach by dividing the dataset into smaller subarrays, sorting them recursively, & then merging them back together.

The final sorted list will help the university identify min., median, & max packages while generating analytical reports.

Sol: ① Approach:-

- Read the no. of package values & store them in an array.
- Apply the merge sort algorithm, which follows the Divide and Conquer approach by recursively dividing the array into smaller subarrays.

- Merge the divided subarrays by comparing their elements & arranging them in ascending order.
- Continue the merging process until the complete array becomes sorted.
- Print the sorted package values in increasing order.

Q8- A large e-commerce tracks the time taken (in min) to process customer orders before shipment. Due to varying warehouse workloads, processing times differ significantly from one order to another. The analytics team wants to study operational efficiency by sorting all processing times & generating statistical information. Since millions of orders are processed every month, the company requires an efficient sorting technique with predictable performance. You'll just use merge sort to arrange all processing times in ascending order.

Sol:- Q8 Approach:-

- Read all processing times and store them in an array.
- Sort the array in ascending order using the Merge sort algorithm.
- After sorting, find the median processing time from the sorted array.
- Count the no. of processing times that are greater than the median & calculate the diff. b/w the max. & min. values.

- Display the sorted array along with the required status file.

Q3. A cloud infrastructure company manages hundreds of servers distributed across different geographical locations. Every hour, each server reports its average response time in milliseconds. The collected data is stored in the order it arrives & is not sorted. To generate performance reports, the operations team wants the response times arranged from the fastest server to the slowest server. Since the data size can become very large, an efficient in-place sorting technique is sought.

Your task is to implement Quick Sort to sort the response times in ascending order. Built-in sorting methods are not allowed.

After sorting, the operations team will use the report to identify high-latency servers & optimize system performance.

Q3 Approach:-

- Read all server response times into an array.
- Use the Quick Sort algorithm by selecting a pivot element & partitioning the array around it.
- Recursively sort the left & right partitions until the entire array is sorted.
- Since Quick Sort works in-place, no extra array is required for sorting.
- Print the response times in ascending order.

Q4 - A financial analytics company receives daily trade value data from multiple stock exchanges. Analysts use this information to identify high-value trading activity & market trends.

The trade values are received in random order throughout the day. Before analysis can begin, the data must be sorted in descending order. Your task is to implement Quick Sort to arrange the trade values from highest to lowest. After sorting, perform the following operations:

1. Display the sorted trade values.
2. Find the Top 5 highest trade values.
3. Calculate the average of the Top 5 values.
4. Count how many trade values are greater than the overall average of all trade values.

This problem evaluates your understanding of partitioning, recursion, and real-world data analysis using Quick Sort.

Sol-① Approach:-

- Read all trade values & store them in an array.
- Apply Quick Sort to arrange the values in descending order by modifying the comparison during partitioning.
- After sorting, display the first five elements as the Top 5 trade values & calculate their average.
- Find the overall average of all trade values & count how many values are greater than this average.
- Display the sorted array & all the calculated results.

Q5 An online gaming platform is hosting a national-level tournament involving thousands of participants. At the end of the tournament, player scores are collected from multiple game servers & stored in random order.

Before announcing the final rankings, the organizers need all scores sorted in ascending order. Since the scoreboard data can be very large, they have decided to use Heap Sort, which provides guaranteed $O(N \log N)$ performance.

Your task is to implement Heap Sort & arrange all player scores in increasing order. The sorted output will help tournament officials generate rankings & determine qualification thresholds for future events.

Approach:-

- Read all player scores into an array.
- Build a Max Heap from the given array.
- Repeatedly swap the root element with the last element & reduce the heap size.
- Heapify the remaining elements after every swap until the entire array becomes sorted.
- Print the player scores in ascending order.

Q6 A multi-speciality hospital maintains records of emergency response times for all critical cases handled during a month. Each response time represents the no. of minutes

taken by the emergency team to reach & begin treatment for a patient.

Hospital administrators want to analyze the efficiency of their emergency services. To do so, all response times must first be sorted using Heap sort. After sorting the response times in ascending order, generate the following statistics:

1. Display the sorted response times.
2. Find the fastest response time.
3. Find the slowest response time.
4. Calculate the average response time.
5. Count the no. of cases handled faster than the average.
6. Calculate the percentage of cases handled faster than the average.

The hospital intends to use this information to improve resource allocation & reduce emergency response delays.

Sol:- ⑥ Approach:-

- Read all emergency response times and store them in an array.
- Sort the response times in ascending order using the Heap Sort algorithm.
- After sorting, find the fastest time, slowest time, & calculate the average response time.
- Count the no. of response times that are less than the average & calculate their percentage.
- Display the sorted response times along with all the required statistics.

```
#include <iostream>
using namespace std;
void merge(int arr[], int s, int e)
{
    int mid = s + (e - s) / 2;
    int len1 = mid - s + 1;
    int len2 = e - mid;
    int *first = new int[len1];
    int *second = new int[len2];
    int mainArrayIndex = s;
    for (int i = 0; i < len1; i++)
    {
        first[i] = arr[mainArrayIndex++];
    }
    mainArrayIndex = mid + 1;
    for (int i = 0; i < len2; i++)
    {
        second[i] = arr[mainArrayIndex++];
    }
    int index1 = 0;
    int index2 = 0;
    mainArrayIndex = s;
    while (index1 < len1 && index2 < len2)
    {
        if (first[index1] < second[index2])
        {
            arr[mainArrayIndex++] = first[index1++];
        }
        else
        {
            arr[mainArrayIndex++] = second[index2++];
        }
    }
}
```

```
,  
while (index1 < len1)  
{  
    arr[mainArrayIndex++] = first[index1++];  
}  
while (index2 < len2)  
{  
    arr[mainArrayIndex++] = second[index2++];  
}  
delete[] first;  
delete[] second;  
}  
void mergeSort(int arr[], int s, int e)  
{  
    if (s >= e)  
        return;  
    int mid = s + (e - s) / 2;  
    mergeSort(arr, s, mid);  
    mergeSort(arr, mid + 1, e);  
    merge(arr, s, e);  
}
```



```

53 }
54 int main()
55 {
56     int n;
57     cin >> n;
58     int arr[n];
59     for (int i = 0; i < n; i++)
60     {
61         cin >> arr[i];
62     }
63     mergeSort(arr, 0, n - 1);
64     cout << "Sorted Package Values: ";
65     for (int i = 0; i < n; i++)
66     {
67         cout << arr[i] << " ";
68     }
69     return 0;
70 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Bhawna\OneDrive\Desktop\C++ Codes> cd "c:\Users\Bhawna\OneDrive\Desktop\

5

450 1200 800 650 300 1500

Sorted Package Values: 300 450 650 800 1200 1500

PS C:\Users\Bhawna\OneDrive\Desktop\C++ Codes> █


```
1  #include <iostream>
2  using namespace std;
3  void merge(int arr[], int s, int e)
4  {
5      int mid = s + (e - s) / 2;
6      int len1 = mid - s + 1;
7      int len2 = e - mid;
8      int *first = new int[len1];
9      int *second = new int[len2];
10     int mainArrayIndex = s;
11     for (int i = 0; i < len1; i++)
12         first[i] = arr[mainArrayIndex++];
13     mainArrayIndex = mid + 1;
14     for (int i = 0; i < len2; i++)
15         second[i] = arr[mainArrayIndex++];
16     int index1 = 0;
17     int index2 = 0;
18     mainArrayIndex = s;
19     while (index1 < len1 && index2 < len2)
20     {
21         if (first[index1] < second[index2])
22             arr[mainArrayIndex++] = first[index1++];
23         else
24             arr[mainArrayIndex++] = second[index2++];
25     }
26     while (index1 < len1)
27         arr[mainArrayIndex++] = first[index1++];
28     while (index2 < len2)
29         arr[mainArrayIndex++] = second[index2++];
30     delete[] first;
31     delete[] second;
32 }
```



```
void mergeSort(int arr[], int s, int e)
{
    if (s >= e)
        return;
    int mid = s + (e - s) / 2;
    mergeSort(arr, s, mid);
    mergeSort(arr, mid + 1, e);
    merge(arr, s, e);
}

int main()
{
    int n;
    cin >> n;
    int arr[n];
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    mergeSort(arr, 0, n - 1);
    cout << "Sorted Processing Times: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
    double median;
    if (n % 2 == 0)
        median = (arr[n / 2] + arr[n / 2 - 1]) / 2.0;
    else
        median = arr[n / 2];

    cout << "Median: " << median << endl;
    int count = 0;
    for (int i = 0; i < n; i++)
    {
        if (arr[i] > median)
            count++;
    }
}
```

```

61     int count = 0;
62     for (int i = 0; i < n; i++)
63     {
64         if (arr[i] > median)
65             count++;
66     }
67     cout << "Orders Above Median: " << count << endl;
68     cout << "Difference: " << arr[n - 1] - arr[0];
69     return 0;
70 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\Bhawna\OneDrive\Desktop\C++ Codes> cd "c:\Users\Bhawna\OneDrive\Desktop\C++ Codes"
7
12 25 18 30 15 22 40
Sorted Processing Times: 12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28
PS C:\Users\Bhawna\OneDrive\Desktop\C++ Codes>

```



```
#include <iostream>
using namespace std;
int partition(int arr[], int s, int e)
{
    int pivot = arr[s];
    int count = 0;
    for (int i = s + 1; i <= e; i++)
    {
        if (arr[i] <= pivot)
            count++;
    }
    int pivotIndex = s + count;
    swap(arr[pivotIndex], arr[s]);
    int i = s;
    int j = e;
    while (i < pivotIndex && j > pivotIndex)
    {
        while (arr[i] <= pivot)
            i++;
        while (arr[j] > pivot)
            j--;
        if (i < pivotIndex && j > pivotIndex)
        {
            swap(arr[i], arr[j]);
            i++;
            j--;
        }
    }
    return pivotIndex;
}
```

```

31 void quickSort(int arr[], int s, int e)
32 {
33     if (s >= e)
34         return;
35     int p = partition(arr, s, e);
36     quickSort(arr, s, p - 1);
37     quickSort(arr, p + 1, e);
38 }
39 int main()
40 {
41     int n;
42     cin >> n;
43     int arr[n];
44     for (int i = 0; i < n; i++)
45         cin >> arr[i];
46     quickSort(arr, 0, n - 1);
47     for (int i = 0; i < n; i++)
48         cout << arr[i] << " ";
49     return 0;
50 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\Bhawna\OneDrive\Desktop\C++ Codes> cd "c:\Users\Bhawna\OneDrive
6
120 45 78 200 60 95
45 60 78 95 120 200
PS C:\Users\Bhawna\OneDrive\Desktop\C++ Codes>

```



```
1  #include <iostream>
2  using namespace std;
3  void merge(int arr[], int s, int e)
4  {
5      int mid = s + (e - s) / 2;
6      int len1 = mid - s + 1;
7      int len2 = e - mid;
8      int *first = new int[len1];
9      int *second = new int[len2];
10     int mainArrayIndex = s;
11     for (int i = 0; i < len1; i++)
12         first[i] = arr[mainArrayIndex++];
13     mainArrayIndex = mid + 1;
14     for (int i = 0; i < len2; i++)
15         second[i] = arr[mainArrayIndex++];
16     int index1 = 0;
17     int index2 = 0;
18     mainArrayIndex = s;
19     while (index1 < len1 && index2 < len2)
20     {
21         if (first[index1] < second[index2])
22             arr[mainArrayIndex++] = first[index1++];
23         else
24             arr[mainArrayIndex++] = second[index2++];
25     }
26     while (index1 < len1)
27         arr[mainArrayIndex++] = first[index1++];
28     while (index2 < len2)
29         arr[mainArrayIndex++] = second[index2++];
30     delete[] first;
31     delete[] second;
32 }
```

```
31 void quickSort(int arr[], int s, int e)
32 {
33     if (s >= e)
34         return;
35     int p = partition(arr, s, e);
36     quickSort(arr, s, p - 1);
37     quickSort(arr, p + 1, e);
38 }
39 int main()
40 {
41     int n;
42     cin >> n;
43     int arr[n];
44     long long totalSum = 0;
45     for (int i = 0; i < n; i++)
46     {
47         cin >> arr[i];
48         totalSum += arr[i];
49     }
50     quickSort(arr, 0, n - 1);
51     cout << "Sorted Trade Values: ";
52     for (int i = 0; i < n; i++)
53         cout << arr[i] << " ";
54     cout << endl;
55     cout << "Top 5: ";
56     long long top5Sum = 0;
57     for (int i = 0; i < 5; i++)
58     {
59         cout << arr[i] << " ";
60         top5Sum += arr[i];
61     }
```



```

62     cout << endl;
63     double top5Average = top5Sum / 5.0;
64     cout << "Average of Top 5: " << top5Average << endl;
65     double overallAverage = totalSum / (double)n;
66     int count = 0;
67     for (int i = 0; i < n; i++)
68     {
69         if (arr[i] > overallAverage)
70             count++;
71     }
72     cout << "Values Above Overall Average: " << count << endl;
73     return 0;
74 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\Bhawna\OneDrive\Desktop\C++ Codes> cd "c:\Users\Bhawna\OneDrive\Desktop\C++ Codes\" ; if
8
500 1200 700 2000 900 1500 3000 2500
Sorted Trade Values: 3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200
Average of Top 5: 2040
Values Above Overall Average: 3
PS C:\Users\Bhawna\OneDrive\Desktop\C++ Codes>

```

```
1  #include <iostream>
2  using namespace std;
3  void heapify(int arr[], int n, int i)
4  {
5      int largest = i;
6      int left = 2 * i + 1;
7      int right = 2 * i + 2;
8      if (left < n && arr[left] > arr[largest])
9          largest = left;
10     if (right < n && arr[right] > arr[largest])
11         largest = right;
12     if (largest != i)
13     {
14         swap(arr[i], arr[largest]);
15         heapify(arr, n, largest);
16     }
17 }
18 void heapSort(int arr[], int n)
19 {
20     for (int i = n / 2 - 1; i >= 0; i--)
21     {
22         heapify(arr, n, i);
23     }
24     for (int i = n - 1; i > 0; i--)
25     {
26         swap(arr[0], arr[i]);
27         heapify(arr, i, 0);
28     }
29 }
```

```
30  int main()
31  {
32      int n;
33      cin >> n;
34      int arr[n];
35      for (int i = 0; i < n; i++)
36          cin >> arr[i];
37      heapSort(arr, n);
38      for (int i = 0; i < n; i++)
39          cout << arr[i] << " ";
40      return 0;
41  }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Bhawna\OneDrive\Desktop\C++ Codes> cd "c:\Users\Bhawna\OneDrive
6
850 1200 950 600 1500 1000
600 850 950 1000 1200 1500
PS C:\Users\Bhawna\OneDrive\Desktop\C++ Codes> █
```



```
1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4  void heapify(int arr[], int n, int i)
5  {
6      int largest = i;
7      int left = 2 * i + 1;
8      int right = 2 * i + 2;
9      if (left < n && arr[left] > arr[largest])
10         largest = left;
11     if (right < n && arr[right] > arr[largest])
12         largest = right;
13     if (largest != i)
14     {
15         swap(arr[i], arr[largest]);
16         heapify(arr, n, largest);
17     }
18 }
19 void heapSort(int arr[], int n)
20 {
21     for (int i = n / 2 - 1; i >= 0; i--)
22     {
23         heapify(arr, n, i);
24     }
25     for (int i = n - 1; i > 0; i--)
26     {
27         swap(arr[0], arr[i]);
28         heapify(arr, i, 0);
29     }
30 }
```

```
31  int main()
32  {
33      int n;
34      cin >> n;
35      int arr[n];
36      int sum = 0;
37      for (int i = 0; i < n; i++)
38      {
39          cin >> arr[i];
40          sum += arr[i];
41      }
42      heapSort(arr, n);
43      cout << "Sorted Response Times: ";
44      for (int i = 0; i < n; i++)
45          cout << arr[i] << " ";
46      cout << endl;
47      cout << "Fastest: " << arr[0] << endl;
48      cout << "Slowest: " << arr[n - 1] << endl;
49      double average = (double)sum / n;
50      cout << "Average: " << average << endl;
51      int count = 0;
```

```

52     for (int i = 0; i < n; i++)
53     {
54         if (arr[i] < average)
55             count++;
56     }
57     cout << "Cases Faster Than Average: " << count << endl;
58     double percentage = (count * 100.0) / n;
59     cout << fixed << setprecision(2) << double percentage
60     cout << "Percentage: " << percentage << "%" << endl;
61     return 0;
62 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\Bhawna\OneDrive\Desktop\C++ Codes> cd "c:\Users\Bhawna\OneDrive\Desktop\C++ Codes\" ;
3
12 7 15 9 20 5 10 8
Sorted Response Times: 5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%
PS C:\Users\Bhawna\OneDrive\Desktop\C++ Codes>

```